Step 6 — Valutazione: un fact checker generale

Obiettivo

Misurare se il modello fine-tunato è un **fact checker migliore in generale**: sulle fonti viste in training, su domini mai visti, su LLM-AggreFact valutato zero-shot, in condizioni di stress, senza aver perso le capacità di decisione generali e con probabilità ben calibrate.

Gli split e le regole sono definiti allo Step 0: qui si descrive come si usano.

1. La suite

Blocco	Dataset	Cosa misura	Quando si guarda	Usato per decidere?
dev_select	~60% del dev delle fonti di training	soglia, temperatura, checkpoint, esperimenti	sempre	**sì**
dev_confirm	~40% del dev, documenti diversi	conferma delle scelte	una volta per fase	solo come conferma
test_id	test delle fonti di training (FEVER `paper_test`, ANLI test, VitaminC test, metà del dev di HoVer e AVeriTeC, 10% di MiniCheck e SummEdits)	quanto il modello ha imparato	una volta per la baseline, una alla fine	no
test_ood	SciFact, Climate-FEVER, HealthVer / COVID-Fact, HaluBench (RAGTruth a parte), HalluMix, LLM-Oasis	generalizzazione a domini mai visti	idem	no
AggreFact test	11 fonti, 29.320 righe	confronto con la leaderboard, **zero-shot**	idem	no
Comportamentali	generati da `dev_select` (§4)	il modello guarda davvero il documento?	a ogni esperimento	come vincolo
Oblio	JevBench pubblico, Kev-v9 clean, typed-decisions test	capacità di decisione generali	a ogni esperimento	come vincolo

Il dev di LLM-AggreFact non entra in nessuna decisione. Si usa solo per la **soglia secondaria** dichiarata (§3).

2. Metriche

Metrica	Definizione	Perché
Balanced accuracy (BAcc)	½ (TPR + TNR), per fonte, poi media	non premia chi dice sempre la classe più frequente
AUC	probabilità che un positivo abbia punteggio più alto di un negativo	indipendente dalla soglia: misura l'ordinamento
Brier	media di $(p - y)^2$	errore delle probabilità

Metrica	Definizione	Perché
ECE (10 intervalli)	scarto medio fra confidenza e accuratezza	calibrazione: "0,9" deve voler dire "giusto 9 volte su 10"
Copertura selettiva	accuratezza sugli esempi con $\lvert p - 0{,}5\rvert > 0{,}3$, e quota di esempi coperti	quanto ci si può fidare delle risposte sicure
Stabilità	deviazione standard della BAcc fra le varianti di domanda tenute da parte	robustezza alla formulazione
Costi	token/s, prompt troncati o falliti	uso reale

Esempio di ECE. 1.000 previsioni con confidenza fra 0,8 e 0,9 (media 0,85), di cui 780 giuste (0,78): contributo = $(1.000 / N) \cdot \lvert 0{,}85 - 0{,}78\rvert$. Se tutte le fasce hanno scarti così, ECE $\approx$ 0,07: il modello è un po' sovra-confidente.

Aggregazione (non si mescolano righe di test diversi in un unico calcolo):

1. BAcc per fonte;
2. media non pesata per blocco: test_id, test_ood, AggreFact (media sulle 11 fonti, come la leaderboard);
3. **numero unico = media dei tre blocchi**;
4. AggreFact riportato anche da solo, nel formato della leaderboard.

Incertezza: intervalli al 95% con bootstrap per documento (`document_cluster_bootstrap` in metrics.py); confronti fra modelli appaiati sugli stessi esempi. Differenze sotto ~0,5 punti di BAcc media vanno considerate non conclusive.

3. Soglia e regola di decisione

- **Una soglia per modello**, fittata su `dev_select` (BAcc macro sulle fonti, griglia 0,01–0,99), dopo la temperatura (minimo log-loss), confermata su `dev_confirm`. La stessa soglia si applica a tutti i test.
- **Soglia secondaria, dichiarata**: fittata sul dev di AggreFact, applicata solo al test di AggreFact, per confrontarsi con i numeri già prodotti (Winnow-BF16 77,42; JPT-9B 76,71). La differenza fra principale e secondaria va riportata: dice quanto la calibrazione del modello dipende dal dominio.
- Niente soglie per fonte, niente nomi di fonte nel prompt.
- Se si usa S3: regola `min(p1, 1 − p2, 1 − p3)` con la stessa soglia unica.
- Dopo il fine-tuning soglia e temperatura vanno **rifittate**: quelle della baseline non valgono più.

Per applicare soglie nuove alle cache esistenti di AggreFact non serve nuova inferenza (stessa logica di `calibrate_transfer.py apply`); serve solo aggiungere la possibilità di leggere soglia e temperatura da un file prodotto sul nostro dev.

4. Test comportamentali (automatici)

Si generano da `dev_select` con trasformazioni semplici, senza annotazione manuale:

Test	Trasformazione	Comportamento atteso	Metrica	Obiettivo indicativo
Solo claim	documento vuoto	non supportato	P(supportato) media	$\ll 0{,}5$ (es. $< 0{,}15$)
Rimozione dell'evidenza	per righe supportate con evidenza nota (FEVER, HoVer), tolte le frasi gold	la probabilità scende	% di righe che passano sotto soglia	$> 80\%$
Distrattori	aggiunti paragrafi irrilevanti prima e dopo	decisione invariata	% di decisioni invariate	$> 95\%$
Posizione	frase di supporto all'inizio, a metà, alla fine di un documento lungo	decisione invariata	BAcc per posizione	differenza < 2 punti

Test	Trasformazione	Comportamento atteso	Metrica	Obiettivo indicativo
Coppie minime	numero, data o entità del claim cambiati (VitaminC, SummEdits, `contradict_claim`)	la decisione si inverte	% di inversioni corrette	più alta della baseline
Formulazione	varianti di domanda mai viste	decisione invariata	deviazione standard della BAcc	più bassa della baseline
Campi rinominati	coppie di nomi tenute da parte (es. `passage` / `assertion`)	decisione invariata	BAcc	vicina a quella con i nomi originali

Esempio di "rimozione dell'evidenza":

> Documento (HoVer): pagina "Nancy R. Howell" + pagina "Saint Paul School of Theology" + ... Claim: "The city of the college that Nancy R. Howell is a Professor in and United Methodist Church of the Resurrection have in common is Kansas." (supportato)
>
> Variante: si toglie la frase "She is Professor of Theology and Philosophy of Religion at Saint Paul School of Theology in Kansas City, Missouri." → il collegamento fra Howell e Kansas City non c'è più: P(supportato) deve scendere sotto la soglia.

Se servirà verificare testi in italiano, aggiungere un sottoinsieme tradotto automaticamente del dev e del test: oggi tutti i dati sono in inglese.

5. Oblio

Benchmark	Riferimento Winnow-BF16 (autori)	Limite accettato
JevBench pubblico (231)	85,28%	calo ≤ 2 punti
Kev-v9 clean (1.046)	81,45%	calo ≤ 2 punti
typed-decisions test (2.000 decisioni)	70,20%	calo ≤ 2 punti
Domande 2 e 3 della regola di Jev	AUC su `dev_select`	nessun calo rispetto alla baseline

Per JPT i riferimenti vengono dalla baseline (01). Gli script ufficiali sono documentati in `winnow-12b/src/docs/EVALUATION.md`; le probabilità non si ricalibrano mai sui risultati di questi benchmark.

6. Protocollo

1. Tutte le scelte su `dev_select`; conferma su `dev_confirm`.
2. I test (test_id, test_ood, AggreFact) si eseguono **una volta per la baseline** e **una volta per il modello finale** congelato (08). Nessuna scelta dopo averli visti.
3. Mai addestrare su LLM-AggreFact (vietato dalla licenza del dataset) né sulle fonti di test_ood.
4. Prima dei test finali, salvare in un `frozen.json` gli hash di pesi fusi, split (manifest dello Step 0), configurazione, soglia e codice di valutazione.
5. Riportare sempre la baseline accanto al modello fine-tunato, con la stessa procedura (stessa soglia "dal nostro dev").

7. Report finale (formato)

Blocco	Metrica	Baseline	Fine-tunato	Δ	IC 95% di Δ
test_id	BAcc macro	...	...	...	...

Blocco	Metrica	Baseline	Fine-tunato	Δ	IC 95% di Δ
test_ood	BAcc macro	…	…	…	…
AggreFact (zero-shot)	BAcc macro / AUC	… / 85,02	…	…	…
Media dei blocchi	BAcc	…	…	…	…
AggreFact (soglia secondaria)	BAcc macro	77,42	…	…	…
Comportamentali	solo claim / rimozione evidenza / …	…	…	…	—
Oblio	JevBench / Kev / typed	85,28 / 81,45 / 70,20	…	…	—
Calibrazione	ECE media	…	…	…	—

Più le tabelle per fonte di ogni blocco e quella di LLM-AggreFact nel formato della leaderboard. I valori della colonna baseline sono quelli di Winnow-BF16 dove già noti.

Codice

Esiste: `jpt-9b/benchmark.py`, `winnow-12b-bf16/benchmark.py` (anche su file con `--data`), `calibrate_transfer.py`, `metrics.py`. Da scrivere: lettura degli split dello Step 0 e opzione per la cartella dei pesi fusi nei benchmark; applicazione di soglie fittate sul nostro dev alle cache di AggreFact; builder delle fonti di test_ood; `fine-tune/behavioral.py` per i test di §4; wrapper per JevBench, Kev e typed-decisions.